Massey University
College of Sciences

161.304: Comments and Solutions for Computer Laboratory 10

Exercise 1: Easy, easy

- 9E1

  Only (3) is required.

- 9E2

  Gibbs sampling is more efficient because it makes smarter proposals. This is accomplished through conjugate priors, which make it possible to get analytical solutions to the posterior for each parameter in the model. However, this is also limiting, because we are constrained in which prior distributions we are able to select. Additionally, like the Metropolis algorithm, Gibbs samplers can get stuck in one area of the posterior if parameters are highly correlated or if the parameters are in a high-dimensional space.

- 9E3

  Hamiltonian Monte Carlo cannot handle discrete parameters. This is because the method uses a physics simulation to glide an imaginary particle over the surface of the posterior. If parameters are discrete, the definition of a gradient becomes problematic; that breaks the surface and the particle cannot glide.

- 9E4

  The actual number of samples is simply the number of draws we make from the posterior distribution. These draws may be autocorrelated, so the effective number of samples is an estimate of the number of completely independent (i.e., uncorrelated) draws that would contain the same amount of information. As autocorrelation increases, the number of effective samples decreases. Usually, `n_eff` will be smaller than the actual number of samples. However, there are times when Stan can make better than independent draws, which would result in an `n_eff` great than the actual number of samples.

- 9E5

  The `Rhat` should approach 1 from above. That is, an `Rhat` greater than 1 will likely indicate a badly behaved chain.

- 9E6

  Good chain:

  ```
  library(rethinking)
  y <- rnorm(100, mean=1, sd=2)
  m1 <- ulam(alist(
    y ~ dnorm(mu, sigma),
    mu <- alpha,
    alpha ~ dnorm(0, 10),
    sigma ~ dcauchy(0,1)
  ), data=list(y=y), start=list(alpha=0, sigma=1), chains=2)
  plot(m1, col=c("black", "royalblue4"))
  traceplot(m1)
  ```

  These chains are doing good: they are stationary, that is, the mean of the chain does not go up or down but the chain stays the whole time between the values 0.5 and 1.5 for `alpha` and between 1.8 and 2.6 for `sigma`.

  Bad chain:

```
y <- rnorm(100, mean=1, sd=2)
m2 <- ulam(alist(
    y ~ dnorm(mu, sigma),
    mu <- a1 + a2,
    c(a1, a2) ~ dnorm(0, 100),
    sigma ~ dcauchy(0,1)
), data=list(y=y), start=list(a1=0, a2=0, sigma=1), chains=2)
plot(m2, col=c("black", "royalblue4"))
traceplot(m2)
```

These chains are not doing well: The chains for a1 and a2 go up and down and don't settle on a mean. While the chains for sigma are somehow closer to each other, they still didn't settle on a mean. We have an identifiability issue, like in Chapter 9. Notice we have seen how to fix this with weakly informative priors.

- 9E7

  Straightforward using the command `trankplot()` instead of `traceplot()`

  A healthy Markov chain should be both *stationary* and `well-mixing`. The first is necessary for inference. The second is desirable, because it means the chain is more efficient. A chain that is both of these things should resemble horizontal noise. A chain that is malfunctioning, as the problem asks, would not be stationary. This means it is not converging to the target distribution, the posterior distribution. Examples were provided in the chapter. A virtue of Hamiltonian Monte Carlo is that it makes such chains very obvious: they tend to be rather flat wandering trends. Sometimes they are perfectly flat. The best test of convergence is always to compare multiple chains. So the best sketch of a malfunctioning trace plot would be one that shows multiple chains wandering into different regions of the parameter space.

Exercise 2: Silly prior, 9M2

We re-prepare the data

```
library(rethinking)
data(rugged)
d <- rugged
d$log_gdp <- log(d$rgdppc_2000)
d <- d[complete.cases(d$rgdppc_2000), ]
d$log_gdp_std <- d$log_gdp / mean(d$log_gdp)
d$rugged_std <- d$rugged / max(d$rugged)
d$cid <- ifelse(d$cont_africa == 1, 1, 2)
dd.trim <- list(
  log_gdp_std = d$log_gdp_std,
  rugged_std = d$rugged_std,
  cid = as.integer(d$cid)
)
```

- 9M1

  Now let's fit two models with an exponential and a uniforma prior for `sigma`:

```
## Exponential prior for sigma
m.M1Exp <- ulam(alist(
    log_gdp_std ~ dnorm(mu, sigma),
    mu <- a[cid] + b[cid] * (rugged_std - 0.215),
    a[cid] ~ dnorm(1, 0.1),
    b[cid] ~ dnorm(0, 0.3),
    sigma ~ dexp(1)
  ), data = dd.trim, chains = 4, cores = 4)
```

```
## Uniform prior for sigma
m.M1Uni <- ulam(alist(
    log_gdp_std ~ dnorm(mu, sigma),
    mu <- a[cid] + b[cid] * (rugged_std - 0.215),
    a[cid] ~ dnorm(1, 0.1),
    b[cid] ~ dnorm(0, 0.3),
    sigma ~ dunif(0,1)
  ), data = dd.trim, chains = 4, cores = 4)
```

Now on to inspect the model. Let's start with the parameter estimates in comparison

```
coeftab(m.M1Exp, m.M1Uni)
```

These are strikingly the same. What about the individual model outputs in more detail?

```
precis(m.M1Exp, depth = 2)
precis(m.M1Uni, depth = 2)
```

Honestly, not much differences, but some variations on `n_eff`, as can be seen by plotting the posterior on `sigma` (do it).

- 9M2

```
m.M2Exp <- ulam(alist(
  log_gdp_std ~ dnorm(mu, sigma),
  mu <- a[cid] + b[cid]*(rugged_std - 0.215),
  a[cid] ~ dnorm(1, 0.2),
  b[cid] ~ dexp(0.3),
  sigma ~ dexp(1)
), data = dd.trim, chains = 4, cores = 4)
precis(m.M2Exp, depth = 2)
```

And now this is quite a change compared to the samples from the non-exponential prior for `b[cid]`.

```
m1.samples <-  extract.samples(m.M1Exp)
plot(density(m1.samples$b[,2]), ylim = c(0, 35))
m2.samples <- extract.samples(m.M2Exp)
lines(density(m2.samples$b[,2]), col = 2)
```

The `b` parameter wants to be negative, as it was in the original model. The new prior constrains the parameter to be positive, so we see all of the posterior density piled up right next to 0.

Exercise 3: An introduction to the Cauchy distribution, 9H1

```
mp <- ulam(alist(
    a ~ dnorm(0, 1),
    b ~ dcauchy(0, 1)
  ), data=list(y=1), chains = 1)
```

The model simply samples from the two distributions: the normal and the Cauchy distribution. See the Stan code and plots for the two distributions:

```
stancode(mp)
plot(mp, col="royalblue4")
traceplot(mp, n_cols = 1)
precis(mp)
```

We can compare the two samples with their exact density function:

```
post <- extract.samples(mp)

par(mfrow=c(1,2))
dens(post$a)
curve(dnorm(x,0,1), from=-4, to=4, add=T, lty=2)
legend("topright", lty=c(1,2), legend=c("Sample", "Exact density"), bty="n")
mtext("Normal")
dens(post$b,  col="royalblue4", xlim=c(-20, 20))
curve(dcauchy(x, 0, 1), from = -20, to=20, add=T, lty=2,
   col="royalblue4")
mtext("Cauchy")
```

While the normal distribution has been approximated very well, the Cauchy distribution has been approximated less well. After all, the number of effective samples for the Cauchy distribution has been relatively small. But visually, we are still quite good!

The `b` parameter looks a little odd (see trace plot). However, this is normal for the Cauchy distribution. This distribution has an undefined moments (mean, variance, etc.) with very heavy tails. Because of this, it will occasionally jump to extreme values.

Exercise 4: Prior cascading effect, 9H3

We re-simulate the data:

```
N <- 100
height <- rnorm(N, 10, 2)
leg_prop <- runif(N, 0.4, 0.5)
leg_left <- leg_prop*height + rnorm(N, 0, 0.02)
leg_right <- leg_prop*height + rnorm(N, 0, 0.02)
d <- data.frame(height, leg_left, leg_right)
```

And now fit the model with `ulam()`:

```
m5.8s <- ulam(alist(
    height ~ dnorm( mu, sigma),
    mu <- a + bl*leg_left + br*leg_right,
    a ~ dnorm(10, 100),
    bl ~ dnorm(2, 10),
    br ~ dnorm(2, 10),
    sigma ~ dexp(1)
  ), data=d, chains=4, start=list(a=10, bl=0, br=0.1, sigma=1))
```

And the second model is:

```
m5.8s2 <- ulam(alist(
    height ~ dnorm( mu, sigma),
    mu <- a + bl*leg_left + br*leg_right,
    a ~ dnorm(10, 100),
    bl ~ dnorm(2, 10),
    br ~ dnorm(2, 10),
    sigma ~ dexp(1)
  ), data=d, chains=4, constraints = list(br = "lower=0"),
  start=list(a=10, bl=0, br=0.1, sigma=1))
```

First, let's have a look at the traceplots:

```
traceplot(m5.8s, n_cols=1)
precis(m5.8s)
traceplot(m5.8s2, n_cols=1)
precis(m5.8s2)
```

The trace plot for the first model looks all good. The estimates and both `Rhat` and `n_eff` don't look too bad. The only thing suspicious here, is that both `bl` and `br` have the same standard deviation.

The pairs plot reveals the problematic correlation between the two parameters:

```
pairs(m5.8s)
pairs(m5.8s2)
```

Whereas before, the two parameter had both a posterior distribution close to normal, now one of them is left-skewed and the other one right-skewed. What happens is that, since both parameter correlate so strongly, we can only reliably estimate their sum. Since we force `br` to be positive, the other part of the sum, `bl`, now more often has to be negative.

If we wanted to fix things, try getting both coefficients non-negative:

```
m5.8s3 <- ulam(alist(
    height ~ dnorm(mu, sigma),
    mu <- a + bl*leg_left + br*leg_right,
    a ~ dnorm(10, 100),
    bl ~ dnorm(2, 10),
    br ~ dnorm(2, 10),
    sigma ~ dexp(1)
  ), data=d, chains=4,
  constraints=list(br="lower=0", bl="lower=0"),
  start=list(a=10,bl=1,br=1,sigma=1))
precis(m5.8s3
pairs(m5.8s3)
```

And this is much better.

Exercise 5: YOUR Metropolis algorithm, 9H6

The model we want to fit can be specified as follow:

$$\begin{aligned} w &\sim \text{Binom}(\theta, \text{n}) \\ \theta &\sim \text{Unif}(0, 1) \end{aligned}$$

```
# the globe tossing data
w <- 6
n <- 9
# prior on p
p_prior <- function(p) dunif(p, min=0, max=1)
# initializing MCMC
iter <- 1e4
p_sample <- rep(0, iter)
p_current <- 0.5       # start value
for (i in 1:iter) {
```

```
  # record current p
  p_sample[i] <- p_current

  # generate proposal
  p_proposal <- runif(1, min=0, max=1)

  # compute likelihood for current and proposal
  lkhd_current <- dbinom(w, n, p_current)
  lkhd_proposal <- dbinom(w, n, p_proposal)

  # assuming a uniform prior of 1 over [0,1],
  # the prior is the same in the numerator and denominator
  # otherwise, multiply times prior at p
  # accept proposal?
  prob_accept <- (lkhd_proposal *p_prior(p_proposal) )/( lkhd_current * p_prior(p_current) )
  p_current <- ifelse( runif(1) < prob_accept, p_proposal, p_current)
}
```

We can visualize the trace plot:

```
plot(p_sample, type="l", col="royalblue4")
```

This chain is fine, but it is more autocorrelated than a typical Stan chain. That's why it tends to wander around a bit. It is stationary, but it isn't mixing very well. This is common of such a simple Metropolis chain. But it is working.

We can also plot the posterior distribution:

```
dens(p_sample, col="royalblue4", adj=1)
curve(dbeta(x, w+1, n-w+1 ), from=0, to=1, add=T, lty=2)
```

The dashed line is the exact analytical solution (see the conjugate Beta-Binomial prior). Our simple MCMC estimator doesn't perform too bad.